```sql
-- ============================================================================
-- ASCEND - CONSULTAS SQL DEL PANEL DE ADMINISTRACION
-- Base de datos: ascend (MySQL 8)
-- ============================================================================
-- Objetivo:
--   Reunir las consultas de lectura que usa el panel de Administrador General
--   (codigo_fuente/modelos/Usuario.php, Equipo.php, Torneo.php, ErrorSistema.php,
--   Juego.php, Formato.php, Newsletter.php, PoliticaContrasena.php, Alerta.php,
--   MensajeContacto.php y SeguridadAuth.php) y las consultas adicionales de
--   auditoria, control e integridad que pueden pedirse en la defensa.
--
-- Politica de roles vigente:
--   - Rol 1 Administrador General: cuentas inmutables desde el panel. No se
--     editan, bloquean ni eliminan desde la interfaz y no se promueve a nadie
--     a rol 1. El alta de administradores se hace fuera de la aplicacion.
--   - Roles asignables desde el panel: 2 Organizador y 3 Jugador.
--   - Rol 4 Usuario Publico: rechazado por el backend y oculto en la interfaz.
--   Las consultas C15, C16, C17 y H01 a H04 permiten auditar esa politica.
--
-- Como usar:
--   Ejecutar sobre la base ascend. Los IDs son ejemplos de la semilla QA:
--   administradores 1 y 1003; organizadores 2, 1000, 1001, 1002, 1004, 1005;
--   jugadores 1010 en adelante; equipos 1100 en adelante; torneos 1200 en
adelante.
--
-- Convencion:
--   Modelo: consulta tomada de un metodo del modelo (adaptada a valores
literales).
--   Extra:  consulta adicional de administracion, auditoria o defensa.
-- ============================================================================

-- USE ascend;

-- ============================================================================
-- BLOQUE A - PANEL, METRICAS Y ULTIMOS MOVIMIENTOS
-- ============================================================================

-- A01. Cantidad de usuarios por rol.
-- Modelo: Usuario::contarTotalJugadoresRegistrados(),
contarTotalOrganizadoresRegistrados()
SELECT r.id AS rol_id,
       r.nombre_rol,
       COUNT(u.id) AS total_usuarios,
       SUM(u.esta_activo = 1) AS activos,
       SUM(u.esta_activo = 0) AS inactivos
FROM roles r
LEFT JOIN usuarios u ON u.rol_id = r.id
GROUP BY r.id, r.nombre_rol
ORDER BY r.id;

-- A02. Resumen general del panel en una sola fila.
-- Extra
```

```sql
SELECT (SELECT COUNT(*) FROM usuarios) AS usuarios,
       (SELECT COUNT(*) FROM usuarios WHERE rol_id = 2) AS organizadores,
       (SELECT COUNT(*) FROM usuarios WHERE rol_id = 3) AS jugadores,
       (SELECT COUNT(*) FROM equipos WHERE activo = 1) AS equipos_activos,
       (SELECT COUNT(*) FROM torneos) AS torneos,
       (SELECT COUNT(*) FROM torneos
        WHERE estado IN ('inscripciones_abiertas', 'en_curso')) AS
torneos_activos,
       (SELECT COUNT(*) FROM torneos WHERE estado = 'finalizado') AS
torneos_finalizados,
       (SELECT COUNT(*) FROM torneos WHERE estado = 'borrador') AS
torneos_borrador,
       (SELECT COUNT(*) FROM participantes_torneo
        WHERE estado = 'pendiente') AS inscripciones_pendientes,
       (SELECT COUNT(*) FROM solicitudes_equipo
        WHERE estado = 'pendiente') AS solicitudes_equipo_pendientes,
       (SELECT COUNT(*) FROM usuarios u
        INNER JOIN perfiles_organizadores po ON po.usuario_id = u.id
        WHERE u.rol_id = 2 AND po.verificado_oficial = 0) AS
organizadores_sin_verificar,
       (SELECT COUNT(*) FROM newsletter_subscriptores WHERE activo = 1) AS
newsletter_activos;

-- A03. Cantidad de torneos activos (inscripciones abiertas o en curso).
-- Modelo: Torneo::contarTorneosActivos()
SELECT COUNT(*) AS torneos_activos
FROM torneos
WHERE estado = 'inscripciones_abiertas'
   OR estado = 'en_curso';

-- A04. Ultimos jugadores registrados.
-- Modelo: Usuario::obtenerUltimosJugadoresRegistrados()
SELECT id, nombre_completo, email, fecha_registro
FROM usuarios
WHERE rol_id = 3
ORDER BY fecha_registro DESC
LIMIT 10;

-- A05. Ultimos organizadores registrados.
-- Modelo: Usuario::obtenerUltimosOrganizadoresRegistrados()
SELECT id, nombre_completo, email, fecha_registro
FROM usuarios
WHERE rol_id = 2
ORDER BY fecha_registro DESC
LIMIT 10;

-- A06. Ultimos equipos creados con cantidad de miembros.
-- Modelo: Equipo::obtenerUltimosEquiposCreados()
SELECT e.id,
       e.nombre_equipo,
       e.creado_por,
       COUNT(em.usuario_id) AS total_miembros
FROM equipos e
LEFT JOIN equipo_miembros em ON em.equipo_id = e.id
```

```sql
GROUP BY e.id, e.nombre_equipo, e.creado_por
ORDER BY e.id DESC
LIMIT 10;

-- A07. Ultimos torneos creados.
-- Modelo: Torneo::obtenerUltimosTorneosCreados()
SELECT id, nombre, estado, fecha_inicio, creado_en
FROM torneos
ORDER BY id DESC
LIMIT 10;

-- A08. Organizadores pendientes de verificacion oficial.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.fecha_registro,
       COALESCE(po.nombre_organizacion, 'Organizador Independiente') AS
organizacion,
       COALESCE(po.verificado_oficial, 0) AS verificado_oficial
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE u.rol_id = 2
  AND COALESCE(po.verificado_oficial, 0) = 0
ORDER BY u.fecha_registro DESC;

-- A09. Solicitudes de ingreso a equipos pendientes de respuesta.
-- Extra
SELECT s.id,
       s.fecha_solicitud,
       e.id AS equipo_id,
       e.nombre_equipo,
       solicitante.nombre_completo AS solicitante,
       iniciador.nombre_completo AS iniciada_por,
       s.mensaje
FROM solicitudes_equipo s
INNER JOIN equipos e ON e.id = s.equipo_id
INNER JOIN usuarios solicitante ON solicitante.id = s.usuario_id
INNER JOIN usuarios iniciador ON iniciador.id = s.iniciado_por
WHERE s.estado = 'pendiente'
ORDER BY s.fecha_solicitud ASC;

-- A10. Inscripciones a torneos pendientes de confirmacion.
-- Extra
SELECT p.id,
       p.fecha_inscripcion,
       t.id AS torneo_id,
       t.nombre AS torneo,
       p.tipo,
       p.nombre AS participante,
       u.nombre_completo AS organizador
FROM participantes_torneo p
INNER JOIN torneos t ON t.id = p.torneo_id
INNER JOIN usuarios u ON u.id = t.organizador_id
```

```sql
WHERE p.estado = 'pendiente'
ORDER BY p.fecha_inscripcion ASC;

-- A11. Cantidad de suscriptores activos al newsletter.
-- Modelo: Newsletter::contarActivos()
SELECT COUNT(*) AS suscriptores_activos
FROM newsletter_subscriptores
WHERE activo = 1;

-- A12. Errores del sistema sin resolver.
-- Extra
SELECT COUNT(*) AS errores_sin_resolver
FROM errores_sistema
WHERE estado IN ('nuevo', 'revision');

-- A13. Notificaciones no leidas en todo el sistema.
-- Extra
SELECT COUNT(*) AS notificaciones_no_leidas
FROM notificaciones
WHERE leido = 0;

-- A14. Actividad registrada en las ultimas 24 horas.
-- Extra
SELECT accion,
       COUNT(*) AS total
FROM logs_actividad
WHERE fecha_hora >= NOW() - INTERVAL 24 HOUR
GROUP BY accion
ORDER BY total DESC;

-- A15. Usuarios registrados en los ultimos 7 dias.
-- Extra
SELECT DATE(fecha_registro) AS dia,
       COUNT(*) AS registrados
FROM usuarios
WHERE fecha_registro >= CURDATE() - INTERVAL 7 DAY
GROUP BY DATE(fecha_registro)
ORDER BY dia;

-- A16. Movimiento reciente de torneos (altas, cambios y bajas).
-- Extra
SELECT id,
       tabla_afectada,
       registro_id,
       accion,
       usuario_id,
       fecha_hora
FROM auditoria_cambios
WHERE tabla_afectada = 'torneos'
ORDER BY fecha_hora DESC, id DESC
LIMIT 30;

-- ============================================================================
-- BLOQUE B - ORGANIZADORES: CONSULTAS PEDIDAS HABITUALMENTE EN LA DEFENSA
```

```sql
-- ============================================================================

-- B01. Listar todos los organizadores registrados.
-- Modelo: Usuario::obtenerTodosUsuariosConRoles(2) + perfiles_organizadores
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.telefono,
       u.esta_activo,
       u.email_verificado,
       u.fecha_registro,
       u.ultimo_acceso,
       COALESCE(NULLIF(po.nombre_organizacion, ''), 'Organizador Independiente')
AS organizacion,
       COALESCE(po.verificado_oficial, 0) AS verificado_oficial,
       po.localidad,
       po.sitio_web
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE u.rol_id = 2
ORDER BY u.nombre_completo;

-- B02. Organizadores con al menos un torneo abierto, finalizado o en borrador.
-- Extra (interpretacion amplia: cualquiera de los tres estados)
SELECT u.id,
       u.nombre_completo,
       u.email,
       COUNT(t.id) AS torneos_en_esos_estados,
       GROUP_CONCAT(DISTINCT t.estado ORDER BY t.estado SEPARATOR ', ') AS estados
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
  AND t.estado IN ('inscripciones_abiertas', 'finalizado', 'borrador')
GROUP BY u.id, u.nombre_completo, u.email
ORDER BY torneos_en_esos_estados DESC, u.nombre_completo;

-- B03. Organizadores registrados con torneos abiertos.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       COUNT(*) AS torneos_abiertos
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
  AND t.estado = 'inscripciones_abiertas'
GROUP BY u.id, u.nombre_completo, u.email
ORDER BY torneos_abiertos DESC, u.nombre_completo;

-- B04. Organizadores registrados con torneos finalizados.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
```

```
       COUNT(*) AS torneos_finalizados
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
  AND t.estado = 'finalizado'
GROUP BY u.id, u.nombre_completo, u.email
ORDER BY torneos_finalizados DESC, u.nombre_completo;

-- B05. Organizadores registrados con torneos en borrador.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       COUNT(*) AS torneos_borrador
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
  AND t.estado = 'borrador'
GROUP BY u.id, u.nombre_completo, u.email
ORDER BY torneos_borrador DESC, u.nombre_completo;

-- B06. Organizadores que tienen simultaneamente torneos abiertos,
-- finalizados y en borrador.
-- Extra (interpretacion estricta: los tres estados a la vez)
SELECT u.id,
       u.nombre_completo,
       u.email,
       SUM(t.estado = 'inscripciones_abiertas') AS abiertos,
       SUM(t.estado = 'finalizado') AS finalizados,
       SUM(t.estado = 'borrador') AS borradores
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
GROUP BY u.id, u.nombre_completo, u.email
HAVING abiertos > 0
   AND finalizados > 0
   AND borradores > 0
ORDER BY u.nombre_completo;

-- B07. Tabla pivote de torneos por organizador y estado (incluye ceros).
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       COALESCE(NULLIF(po.nombre_organizacion, ''), 'Organizador Independiente')
AS organizacion,
       COUNT(t.id) AS total_torneos,
       COALESCE(SUM(t.estado = 'inscripciones_abiertas'), 0) AS abiertos,
       COALESCE(SUM(t.estado = 'en_curso'), 0) AS en_curso,
       COALESCE(SUM(t.estado = 'finalizado'), 0) AS finalizados,
       COALESCE(SUM(t.estado = 'borrador'), 0) AS borradores,
       COALESCE(SUM(t.estado = 'cancelado'), 0) AS cancelados
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
```

```sql
LEFT JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
GROUP BY u.id, u.nombre_completo, u.email, po.nombre_organizacion
ORDER BY total_torneos DESC, u.nombre_completo;

-- B08. Organizadores registrados que no tienen ningun torneo.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.fecha_registro
FROM usuarios u
LEFT JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
  AND t.id IS NULL
ORDER BY u.nombre_completo;

-- B09. Organizador con mas torneos creados.
-- Extra
SELECT u.id,
       u.nombre_completo,
       COUNT(t.id) AS total_torneos
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id = 2
GROUP BY u.id, u.nombre_completo
ORDER BY total_torneos DESC, u.nombre_completo
LIMIT 10;

-- B10. Ranking de organizadores por participantes confirmados.
-- Extra
SELECT u.id,
       u.nombre_completo,
       COUNT(DISTINCT t.id) AS torneos,
       COUNT(p.id) AS participantes_confirmados
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
LEFT JOIN participantes_torneo p
       ON p.torneo_id = t.id
      AND p.estado = 'confirmado'
WHERE u.rol_id = 2
GROUP BY u.id, u.nombre_completo
ORDER BY participantes_confirmados DESC, u.nombre_completo;

-- B11. Ocupacion de los torneos de cada organizador.
-- Extra
SELECT u.id AS organizador_id,
       u.nombre_completo AS organizador,
       t.id AS torneo_id,
       t.nombre AS torneo,
       t.estado,
       t.cupo_max_equipos,
       COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END) AS confirmados,
       ROUND(
```

```
            COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END)
            / t.cupo_max_equipos * 100, 1
        ) AS ocupacion_pct
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
WHERE u.rol_id = 2
GROUP BY u.id, u.nombre_completo, t.id, t.nombre, t.estado, t.cupo_max_equipos
ORDER BY u.nombre_completo, ocupacion_pct DESC;

-- B12. Organizadores con inscripciones pendientes de responder.
-- Extra
SELECT u.id AS organizador_id,
       u.nombre_completo AS organizador,
       t.id AS torneo_id,
       t.nombre AS torneo,
       COUNT(p.id) AS pendientes
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
INNER JOIN participantes_torneo p ON p.torneo_id = t.id AND p.estado = 'pendiente'
WHERE u.rol_id = 2
GROUP BY u.id, u.nombre_completo, t.id, t.nombre
ORDER BY pendientes DESC, u.nombre_completo;

-- B13. Organizadores que crearon equipos (auditoria de datos).
-- Extra (el alta de equipos solo esta habilitada para rol Jugador; esta consulta
-- detecta equipos historicos o de semilla cuyo creador hoy es organizador/admin)
SELECT u.id AS creador_id,
       u.nombre_completo,
       r.nombre_rol,
       COUNT(e.id) AS equipos_creados,
       GROUP_CONCAT(e.nombre_equipo ORDER BY e.id SEPARATOR ' | ') AS equipos
FROM equipos e
INNER JOIN usuarios u ON u.id = e.creado_por
INNER JOIN roles r ON r.id = u.rol_id
WHERE u.rol_id IN (1, 2)
GROUP BY u.id, u.nombre_completo, r.nombre_rol
ORDER BY equipos_creados DESC;

-- B14. Usuarios que organizan torneos aunque su rol actual no sea Organizador.
-- Extra (control de cambios de rol y datos historicos)
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.rol_id,
       r.nombre_rol,
       COUNT(t.id) AS torneos_organizados
FROM usuarios u
INNER JOIN roles r ON r.id = u.rol_id
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id <> 2
GROUP BY u.id, u.nombre_completo, u.email, u.rol_id, r.nombre_rol
ORDER BY torneos_organizados DESC;
```

```sql
-- B15. Estado de verificacion de todos los organizadores.
-- Extra
SELECT COALESCE(po.verificado_oficial, 0) AS verificado,
       COUNT(*) AS organizadores
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE u.rol_id = 2
GROUP BY COALESCE(po.verificado_oficial, 0)
ORDER BY verificado DESC;

-- B16. Organizadores sin perfil en perfiles_organizadores.
-- Extra
SELECT u.id, u.nombre_completo, u.email, u.fecha_registro
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE u.rol_id = 2
  AND po.usuario_id IS NULL
ORDER BY u.id;

-- B17. Organizadores con torneos en curso y partidos sin finalizar.
-- Extra
SELECT u.id AS organizador_id,
       u.nombre_completo AS organizador,
       t.id AS torneo_id,
       t.nombre AS torneo,
       COUNT(e.id) AS partidos_sin_finalizar
FROM usuarios u
INNER JOIN torneos t ON t.organizador_id = u.id
INNER JOIN torneo_encuentros e ON e.torneo_id = t.id AND e.estado <> 'finalizado'
WHERE u.rol_id = 2
  AND t.estado = 'en_curso'
GROUP BY u.id, u.nombre_completo, t.id, t.nombre
ORDER BY partidos_sin_finalizar DESC;

-- B18. Datos de contacto de los organizadores.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.telefono,
       po.telefono_contacto,
       po.sitio_web,
       po.localidad,
       COALESCE(po.verificado_oficial, 0) AS verificado_oficial
FROM usuarios u
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE u.rol_id = 2
ORDER BY u.nombre_completo;

-- ============================================================================
-- BLOQUE C - USUARIOS, ROLES, SEGURIDAD DE CUENTAS Y AUDITORIA
-- ============================================================================

-- C01. Directorio completo de usuarios con rol y perfil de organizador.
```

```sql
-- Modelo: Usuario::obtenerTodosUsuariosConRoles()
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.telefono,
       u.esta_activo,
       u.email_verificado,
       u.fecha_registro,
       u.rol_id,
       r.nombre_rol,
       COALESCE(po.nombre_organizacion, 'Organizador Independiente') AS
nombre_organizacion,
       COALESCE(po.verificado_oficial, 0) AS verificado_oficial
FROM usuarios u
INNER JOIN roles r ON u.rol_id = r.id
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
ORDER BY u.id DESC;

-- C02. Directorio filtrado por rol Organizador.
-- Modelo: Usuario::obtenerTodosUsuariosConRoles(2)
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.telefono,
       u.esta_activo,
       u.fecha_registro,
       u.rol_id,
       r.nombre_rol,
       COALESCE(po.nombre_organizacion, 'Organizador Independiente') AS
nombre_organizacion,
       COALESCE(po.verificado_oficial, 0) AS verificado_oficial
FROM usuarios u
INNER JOIN roles r ON u.rol_id = r.id
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE u.rol_id = 2
ORDER BY u.id DESC;

-- C03. Directorio filtrado por rol Jugador.
-- Modelo: Usuario::obtenerTodosUsuariosConRoles(3)
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.telefono,
       u.esta_activo,
       u.fecha_registro,
       u.rol_id,
       r.nombre_rol
FROM usuarios u
INNER JOIN roles r ON u.rol_id = r.id
WHERE u.rol_id = 3
ORDER BY u.id DESC;

-- C04. Buscar usuarios por nombre o correo.
-- Extra
```

```sql
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.rol_id,
       r.nombre_rol,
       u.esta_activo
FROM usuarios u
INNER JOIN roles r ON r.id = u.rol_id
WHERE u.nombre_completo LIKE '%rodriguez%'
   OR u.email LIKE '%rodriguez%'
ORDER BY u.nombre_completo
LIMIT 50;

-- C05. Ficha administrativa de un usuario.
-- Modelo: Usuario::obtenerFichaAdmin()
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.telefono,
       u.foto_perfil_url,
       u.rol_id,
       u.esta_activo,
       u.email_verificado,
       u.fecha_registro,
       u.ultimo_acceso,
       r.nombre_rol
FROM usuarios u
INNER JOIN roles r ON r.id = u.rol_id
WHERE u.id = 1010
LIMIT 1;

-- C06. Perfil de jugador visto por el administrador.
-- Modelo: Usuario::obtenerPerfilJugadorAdmin()
SELECT apodo_gamertag,
       bio,
       banner_url,
       nivel,
       experiencia_puntos,
       pais,
       ciudad,
       fecha_nacimiento,
       discord_tag,
       instagram_url,
       twitter_url,
       campos_publicos,
       actualizado_en
FROM perfiles_jugadores
WHERE usuario_id = 1010
LIMIT 1;

-- C07. Perfil de organizador visto por el administrador.
-- Modelo: Usuario::obtenerPerfilOrganizadorAdmin()
SELECT nombre_organizacion,
       bio_organizacion,
```

```
       banner_url,
       localidad,
       telefono_contacto,
       sitio_web,
       verificado_oficial,
       campos_publicos,
       actualizado_en
FROM perfiles_organizadores
WHERE usuario_id = 1000
LIMIT 1;

-- C08. Aceptaciones legales de un usuario.
-- Modelo: Usuario::obtenerAceptacionesLegales()
SELECT tipo_documento,
       version,
       aceptado_en,
       ip_origen,
       user_agent
FROM aceptaciones_legales
WHERE usuario_id = 1010
ORDER BY tipo_documento ASC, aceptado_en DESC, id DESC;

-- C09. Resumen de accesos de un usuario.
-- Modelo: Usuario::obtenerResumenAccesos()
SELECT COUNT(*) AS total,
       COALESCE(SUM(exito = 1), 0) AS exitosos,
       COALESCE(SUM(exito = 0), 0) AS fallidos,
       MAX(CASE WHEN exito = 0 THEN fecha_hora END) AS ultimo_fallido
FROM logs_acceso
WHERE usuario_id = 1010;

-- C10. Accesos recientes de un usuario.
-- Modelo: Usuario::obtenerAccesosRecientes()
SELECT id,
       exito,
       ip_origen,
       user_agent,
       mensaje_error,
       fecha_hora
FROM logs_acceso
WHERE usuario_id = 1010
ORDER BY fecha_hora DESC, id DESC
LIMIT 20;

-- C11. Sesiones activas de un usuario.
-- Modelo: Usuario::obtenerSesionesAdmin()
SELECT id,
       ip_origen,
       user_agent,
       ultima_actividad,
       creado_en,
       expira_en,
       activa
FROM sesiones_activas
```

```sql
WHERE usuario_id = 1010
ORDER BY activa DESC, ultima_actividad DESC
LIMIT 10;

-- C12. Actividad reciente de un usuario.
-- Modelo: Usuario::obtenerActividadReciente()
SELECT id,
       accion,
       descripcion,
       ip_origen,
       fecha_hora
FROM logs_actividad
WHERE usuario_id = 1010
ORDER BY fecha_hora DESC, id DESC
LIMIT 20;

-- C13. Metricas de un usuario.
-- Modelo: Usuario::obtenerMetricasUsuario()
SELECT (SELECT COUNT(*) FROM equipos WHERE creado_por = 1010) AS equipos_creados,
       (SELECT COUNT(*) FROM torneos WHERE organizador_id = 1010) AS
torneos_organizados,
       (SELECT COUNT(*) FROM equipo_miembros
        WHERE usuario_id = 1010 AND es_activo = 1) AS equipos_activos;

-- C14. Impacto de un cambio de rol sobre torneos, equipos y capitanias.
-- Modelo: Usuario::obtenerImpactoRol()
SELECT (SELECT COUNT(*) FROM torneos WHERE organizador_id = 1000) AS torneos,
       (SELECT COUNT(*) FROM equipos WHERE creado_por = 1000) AS equipos,
       (SELECT COUNT(*) FROM equipo_capitanes WHERE usuario_id = 1000) AS
capitanias;

-- C15. Cuentas de Administrador General (cuentas protegidas e inmutables).
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.esta_activo,
       u.email_verificado,
       u.fecha_registro,
       u.ultimo_acceso
FROM usuarios u
WHERE u.rol_id = 1
ORDER BY u.id;

-- C16. Cuentas con rol 4 Usuario Publico (no deberia haber ninguna).
-- Extra (la politica vigente rechaza ese rol)
SELECT id, nombre_completo, email, esta_activo, fecha_registro
FROM usuarios
WHERE rol_id = 4
ORDER BY id;

-- C17. Usuarios cuyo rol no coincide con sus perfiles especializados.
-- Extra (control de la herencia 1:1 y de cambios de rol)
SELECT u.id,
```

```
       u.nombre_completo,
       u.rol_id,
       r.nombre_rol,
       pj.usuario_id AS tiene_perfil_jugador,
       po.usuario_id AS tiene_perfil_organizador
FROM usuarios u
INNER JOIN roles r ON r.id = u.rol_id
LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE (u.rol_id = 3 AND pj.usuario_id IS NULL)
   OR (u.rol_id = 2 AND po.usuario_id IS NULL)
   OR (u.rol_id NOT IN (2, 3)
       AND (pj.usuario_id IS NOT NULL OR po.usuario_id IS NOT NULL))
ORDER BY u.id;

-- C18. Cuentas activas con correo sin verificar.
-- Extra
SELECT id, nombre_completo, email, rol_id, fecha_registro, ultimo_acceso
FROM usuarios
WHERE esta_activo = 1
  AND email_verificado = 0
ORDER BY fecha_registro DESC;

-- C19. Usuarios activos a los que les falta aceptar terminos o privacidad.
-- Extra
SELECT x.id,
       x.nombre_completo,
       x.email,
       x.rol_id,
       x.acepto_terminos,
       x.acepto_privacidad
FROM (
    SELECT u.id,
           u.nombre_completo,
           u.email,
           u.rol_id,
           EXISTS (SELECT 1 FROM aceptaciones_legales a
                   WHERE a.usuario_id = u.id
                     AND a.tipo_documento = 'terminos') AS acepto_terminos,
           EXISTS (SELECT 1 FROM aceptaciones_legales a
                   WHERE a.usuario_id = u.id
                     AND a.tipo_documento = 'privacidad') AS acepto_privacidad
    FROM usuarios u
    WHERE u.esta_activo = 1
) x
WHERE x.acepto_terminos = 0
   OR x.acepto_privacidad = 0
ORDER BY x.id;

-- C20. Usuarios con mas de una sesion activa.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
```

```
       COUNT(s.id) AS sesiones_activas
FROM usuarios u
INNER JOIN sesiones_activas s ON s.usuario_id = u.id AND s.activa = 1
GROUP BY u.id, u.nombre_completo, u.email
HAVING COUNT(s.id) > 1
ORDER BY sesiones_activas DESC;

-- C21. Fallos de inicio de sesion en las ultimas 24 horas.
-- Extra
SELECT id,
       email_intentado,
       usuario_id,
       ip_origen,
       mensaje_error,
       fecha_hora
FROM logs_acceso
WHERE exito = 0
  AND fecha_hora >= NOW() - INTERVAL 24 HOUR
ORDER BY fecha_hora DESC;

-- C22. Direcciones IP bloqueadas por intentos fallidos.
-- Modelo: SeguridadAuth::estaIpBloqueada()
SELECT ip_origen,
       intentos,
       ultimo_intento,
       bloqueado_hasta
FROM intentos_login
WHERE bloqueado_hasta IS NOT NULL
  AND bloqueado_hasta > NOW()
ORDER BY bloqueado_hasta DESC;

-- C23. Politica de contrasenas vigente.
-- Modelo: PoliticaContrasena::obtenerPoliticaVigente()
SELECT p.*,
       u.nombre_completo AS actualizado_por_nombre,
       u.email AS actualizado_por_email
FROM politicas_contrasenas p
LEFT JOIN usuarios u ON p.actualizado_por = u.id
WHERE p.id = 1
LIMIT 1;

-- C24. Historial de cambios de contrasena por usuario.
-- Extra
SELECT h.usuario_id,
       u.nombre_completo,
       COUNT(*) AS cambios_registrados,
       MAX(h.fecha_cambio) AS ultimo_cambio
FROM historial_contrasenas h
INNER JOIN usuarios u ON u.id = h.usuario_id
GROUP BY h.usuario_id, u.nombre_completo
ORDER BY ultimo_cambio DESC;

-- C25. Auditoria de cambios de rol y estado de usuarios.
-- Extra (acciones registradas por el panel en logs_actividad)
```

```sql
SELECT l.id,
       l.fecha_hora,
       l.usuario_id,
       u.nombre_completo AS administrador,
       l.accion,
       l.descripcion,
       l.ip_origen
FROM logs_actividad l
LEFT JOIN usuarios u ON u.id = l.usuario_id
WHERE l.accion IN (
    'ADMIN_ACTUALIZA_USUARIO',
    'ADMIN_CAMBIA_ESTADO_USUARIO',
    'ADMIN_ELIMINA_USUARIO'
)
ORDER BY l.fecha_hora DESC, l.id DESC;

-- C26. Auditoria del modo suplantacion (Ver como).
-- Extra
SELECT l.id,
       l.fecha_hora,
       l.usuario_id,
       u.nombre_completo AS administrador,
       l.accion,
       l.descripcion,
       l.ip_origen
FROM logs_actividad l
LEFT JOIN usuarios u ON u.id = l.usuario_id
WHERE l.accion IN ('ADMIN_SUPLANTA_INICIO', 'ADMIN_SUPLANTA_FIN')
ORDER BY l.fecha_hora DESC, l.id DESC;

-- C27. Usuarios dados de baja logica que conservan datos relacionados.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.rol_id,
       u.fecha_registro,
       (SELECT COUNT(*) FROM torneos WHERE organizador_id = u.id) AS torneos,
       (SELECT COUNT(*) FROM equipos WHERE creado_por = u.id) AS equipos_creados,
       (SELECT COUNT(*) FROM equipo_miembros
        WHERE usuario_id = u.id AND es_activo = 1) AS equipos_activos
FROM usuarios u
WHERE u.esta_activo = 0
ORDER BY u.id;

-- C28. Roles existentes y nivel de permiso.
-- Modelo: Usuario::obtenerTodosLosRoles()
SELECT id, nombre_rol, descripcion, nivel_permiso
FROM roles
ORDER BY id ASC;

-- C29. Permisos asignados a cada rol (tablas RBAC).
-- Extra
SELECT r.id AS rol_id,
```

```
       r.nombre_rol,
       p.nombre_permiso,
       rp.asignado_en
FROM roles r
LEFT JOIN rol_permisos rp ON rp.rol_id = r.id
LEFT JOIN permisos p ON p.id = rp.permiso_id
ORDER BY r.id, p.nombre_permiso;

-- C30. Usuarios con actividad administrativa reciente.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       COUNT(l.id) AS acciones_registradas,
       MAX(l.fecha_hora) AS ultima_accion
FROM usuarios u
INNER JOIN logs_actividad l ON l.usuario_id = u.id
WHERE u.rol_id = 1
GROUP BY u.id, u.nombre_completo, u.email
ORDER BY acciones_registradas DESC;

-- ============================================================================
-- BLOQUE D - EQUIPOS DESDE LA ADMINISTRACION
-- ============================================================================

-- D01. Directorio administrativo de equipos.
-- Modelo: Equipo::obtenerDirectorioAdmin()
SELECT e.id,
       e.nombre_equipo,
       e.fecha_creacion,
       e.activo,
       e.escudo_url,
       u.nombre_completo AS lider_nombre,
       (SELECT COUNT(*)
        FROM equipo_miembros em
        WHERE em.equipo_id = e.id AND em.es_activo = 1) AS total_miembros
FROM equipos e
INNER JOIN usuarios u ON u.id = e.creado_por
ORDER BY e.fecha_creacion DESC;

-- D02. Directorio administrativo filtrado por juego.
-- Modelo: Equipo::obtenerDirectorioAdmin($juegoId, 'nuevos')
SELECT e.id,
       e.nombre_equipo,
       e.fecha_creacion,
       e.activo,
       e.escudo_url,
       u.nombre_completo AS lider_nombre,
       (SELECT COUNT(*)
        FROM equipo_miembros em
        WHERE em.equipo_id = e.id AND em.es_activo = 1) AS total_miembros
FROM equipos e
INNER JOIN usuarios u ON u.id = e.creado_por
WHERE e.juego_id = 1
```

```sql
ORDER BY e.fecha_creacion DESC;

-- D03. Ficha administrativa de un equipo.
-- Modelo: Equipo::obtenerFichaAdmin()
SELECT e.*,
       j.nombre AS nombre_juego,
       j.imagen_url AS juego_icono,
       u.nombre_completo AS creador_nombre,
       u.email AS creador_email
FROM equipos e
LEFT JOIN juegos j ON j.id = e.juego_id
INNER JOIN usuarios u ON u.id = e.creado_por
WHERE e.id = 1100
LIMIT 1;

-- D04. Solicitudes pendientes de un equipo.
-- Modelo: Equipo::obtenerFichaAdmin() (segunda consulta)
SELECT COUNT(*) AS solicitudes_pendientes
FROM solicitudes_equipo
WHERE equipo_id = 1100
  AND estado = 'pendiente';

-- D05. Miembros de un equipo con estado de membresia.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       em.posicion,
       em.numero_camiseta,
       em.es_activo,
       em.fecha_union
FROM equipo_miembros em
INNER JOIN usuarios u ON u.id = em.usuario_id
WHERE em.equipo_id = 1100
ORDER BY em.es_activo DESC, u.nombre_completo;

-- D06. Capitanes de un equipo y quien los designo.
-- Extra
SELECT u.id AS capitan_id,
       u.nombre_completo AS capitan,
       ec.es_capitan_principal,
       ec.fecha_asignacion,
       asignador.id AS asignado_por_id,
       asignador.nombre_completo AS asignado_por
FROM equipo_capitanes ec
INNER JOIN usuarios u ON u.id = ec.usuario_id
INNER JOIN usuarios asignador ON asignador.id = ec.asignado_por
WHERE ec.equipo_id = 1100
ORDER BY ec.es_capitan_principal DESC, ec.fecha_asignacion;

-- D07. Participaciones y posiciones de un equipo.
-- Modelo: Equipo::obtenerParticipacionesAdmin()
SELECT pt.id,
       pt.torneo_id,
```

```
        pt.nombre AS nombre_participante,
        pt.estado AS estado_inscripcion,
        pt.fecha_inscripcion,
        t.nombre AS torneo_nombre,
        t.estado AS torneo_estado,
        t.formato,
        t.fecha_inicio,
        t.fecha_fin,
        pos.partidos_jugados,
        pos.partidos_ganados,
        pos.partidos_empatados,
        pos.partidos_perdidos,
        pos.puntos,
        pos.puntos_favor,
        pos.puntos_contra,
        pos.diferencia_goles,
        CASE WHEN pos.participante_id IS NULL THEN NULL ELSE (
            SELECT COUNT(*) + 1
            FROM torneo_posiciones p2
            WHERE p2.torneo_id = pt.torneo_id
              AND (
                  p2.puntos > pos.puntos
                  OR (p2.puntos = pos.puntos AND p2.diferencia_goles >
pos.diferencia_goles)
                  OR (p2.puntos = pos.puntos AND p2.diferencia_goles =
pos.diferencia_goles
                      AND p2.puntos_favor > pos.puntos_favor)
              )
        ) END AS posicion
FROM participantes_torneo pt
INNER JOIN torneos t ON t.id = pt.torneo_id
LEFT JOIN torneo_posiciones pos
       ON pos.torneo_id = pt.torneo_id
      AND pos.participante_id = pt.id
WHERE pt.tipo = 'equipo'
  AND pt.referencia_id = 1100
ORDER BY t.fecha_inicio DESC, t.id DESC;

-- D08. Partidos de un equipo.
-- Modelo: Equipo::obtenerPartidosAdmin()
SELECT enc.id,
       enc.torneo_id,
       t.nombre AS torneo_nombre,
       enc.ronda,
       enc.fecha_hora_programada,
       enc.estado,
       enc.resultado_local,
       enc.resultado_visitante,
       IF(enc.participante_local_id = pt.id, 'Local', 'Visitante') AS condicion,
       COALESCE(rival.nombre, 'Sin rival') AS rival_nombre,
       CASE
           WHEN enc.estado = 'finalizado' AND enc.participante_ganador_id = pt.id
THEN 'V'
           WHEN enc.estado = 'finalizado' AND enc.participante_ganador_id IS NULL
```

```
THEN 'E'
           WHEN enc.estado = 'finalizado' THEN 'D'
           ELSE 'P'
       END AS resultado
FROM torneo_encuentros enc
INNER JOIN torneos t ON t.id = enc.torneo_id
INNER JOIN participantes_torneo pt
        ON pt.torneo_id = enc.torneo_id
       AND pt.tipo = 'equipo'
       AND pt.referencia_id = 1100
       AND (enc.participante_local_id = pt.id OR enc.participante_visitante_id =
pt.id)
LEFT JOIN participantes_torneo rival
       ON rival.id = IF(enc.participante_local_id = pt.id,
                        enc.participante_visitante_id,
                        enc.participante_local_id)
ORDER BY COALESCE(enc.fecha_hora_programada, enc.creado_en) DESC, enc.id DESC
LIMIT 50;

-- D09. Equipos cuyo creador ya no es jugador.
-- Extra
SELECT e.id,
       e.nombre_equipo,
       e.fecha_creacion,
       u.id AS creador_id,
       u.nombre_completo AS creador,
       r.nombre_rol
FROM equipos e
INNER JOIN usuarios u ON u.id = e.creado_por
INNER JOIN roles r ON r.id = u.rol_id
WHERE u.rol_id <> 3
ORDER BY e.fecha_creacion DESC;

-- D10. Equipos sin miembros activos.
-- Extra
SELECT e.id, e.nombre_equipo, e.activo, e.fecha_creacion
FROM equipos e
LEFT JOIN equipo_miembros em ON em.equipo_id = e.id AND em.es_activo = 1
WHERE em.usuario_id IS NULL
ORDER BY e.id;

-- D11. Equipos sin capitan.
-- Extra
SELECT e.id, e.nombre_equipo, e.activo, e.fecha_creacion
FROM equipos e
LEFT JOIN equipo_capitanes ec ON ec.equipo_id = e.id
WHERE ec.equipo_id IS NULL
ORDER BY e.id;

-- D12. Equipos con mas de un capitan principal.
-- Extra
SELECT ec.equipo_id,
       e.nombre_equipo,
       COUNT(*) AS capitanes_principales
```

```sql
FROM equipo_capitanes ec
INNER JOIN equipos e ON e.id = ec.equipo_id
WHERE ec.es_capitan_principal = 1
GROUP BY ec.equipo_id, e.nombre_equipo
HAVING COUNT(*) > 1;

-- D13. Capitanes que no son miembros activos.
-- Extra
SELECT ec.equipo_id,
       e.nombre_equipo,
       u.id AS capitan_id,
       u.nombre_completo AS capitan,
       ec.es_capitan_principal,
       em.es_activo AS membresia_activa
FROM equipo_capitanes ec
INNER JOIN equipos e ON e.id = ec.equipo_id
INNER JOIN usuarios u ON u.id = ec.usuario_id
LEFT JOIN equipo_miembros em
       ON em.equipo_id = ec.equipo_id
      AND em.usuario_id = ec.usuario_id
WHERE em.usuario_id IS NULL
   OR em.es_activo = 0
ORDER BY ec.equipo_id;

-- D14. Equipos con solicitudes pendientes.
-- Extra
SELECT e.id,
       e.nombre_equipo,
       COUNT(s.id) AS solicitudes_pendientes
FROM equipos e
INNER JOIN solicitudes_equipo s ON s.equipo_id = e.id AND s.estado = 'pendiente'
GROUP BY e.id, e.nombre_equipo
ORDER BY solicitudes_pendientes DESC, e.nombre_equipo;

-- D15. Cantidad de equipos por juego y estado.
-- Extra
SELECT j.id,
       j.nombre AS juego,
       COUNT(e.id) AS total_equipos,
       SUM(e.activo = 1) AS activos,
       SUM(e.activo = 0) AS inactivos
FROM juegos j
LEFT JOIN equipos e ON e.juego_id = j.id
GROUP BY j.id, j.nombre
ORDER BY total_equipos DESC, j.nombre;

-- D16. Buscar equipos por nombre.
-- Extra
SELECT e.id,
       e.nombre_equipo,
       e.activo,
       u.nombre_completo AS creador
FROM equipos e
INNER JOIN usuarios u ON u.id = e.creado_por
```

```sql
WHERE e.nombre_equipo LIKE '%gaming%'
ORDER BY e.nombre_equipo;

-- D17. Verificar si un nombre de equipo ya esta en uso (antes de renombrar).
-- Modelo: Equipo::renombrarEquipoAdmin() y equipoNombreEnUso()
SELECT id, nombre_equipo
FROM equipos
WHERE nombre_equipo = 'Nocturnos eSports'
  AND id <> 1100
LIMIT 1;

-- D18. Impacto de renombrar un equipo: participaciones y podios afectados.
-- Modelo: Equipo::renombrarEquipoAdmin() (consultas de impacto)
SELECT DISTINCT torneo_id
FROM participantes_torneo
WHERE tipo = 'equipo'
  AND referencia_id = 1100;

SELECT c.torneo_id,
       c.valor AS podio_actual
FROM torneo_config c
INNER JOIN participantes_torneo p
        ON p.torneo_id = c.torneo_id
       AND p.tipo = 'equipo'
       AND p.referencia_id = 1100
WHERE c.clave = 'podio';

-- D19. Resumen de torneos y posiciones de un equipo.
-- Extra
SELECT t.id AS torneo_id,
       t.nombre AS torneo,
       t.estado,
       pt.estado AS inscripcion,
       pos.partidos_jugados,
       pos.partidos_ganados,
       pos.puntos,
       pos.diferencia_goles
FROM participantes_torneo pt
INNER JOIN torneos t ON t.id = pt.torneo_id
LEFT JOIN torneo_posiciones pos
       ON pos.torneo_id = pt.torneo_id
      AND pos.participante_id = pt.id
WHERE pt.tipo = 'equipo'
  AND pt.referencia_id = 1100
ORDER BY t.fecha_inicio DESC, t.id DESC;

-- D20. Equipos con miembros que compiten en dos equipos del mismo torneo.
-- Extra (control de la regla de negocio)
SELECT t.id AS torneo_id,
       t.nombre AS torneo,
       u.id AS usuario_id,
       u.nombre_completo,
       COUNT(DISTINCT e.id) AS equipos,
       GROUP_CONCAT(DISTINCT e.nombre_equipo ORDER BY e.nombre_equipo SEPARATOR '
```

```
| ') AS nombres_equipos
FROM usuarios u
INNER JOIN equipo_miembros em ON em.usuario_id = u.id AND em.es_activo = 1
INNER JOIN equipos e ON e.id = em.equipo_id AND e.activo = 1
INNER JOIN participantes_torneo p
        ON p.tipo = 'equipo'
       AND p.referencia_id = e.id
       AND p.estado IN ('confirmado', 'pendiente')
INNER JOIN torneos t ON t.id = p.torneo_id
GROUP BY t.id, t.nombre, u.id, u.nombre_completo
HAVING COUNT(DISTINCT e.id) > 1
ORDER BY t.id, u.nombre_completo;

-- ============================================================================
-- BLOQUE E - TORNEOS DESDE LA ADMINISTRACION
-- ============================================================================

-- E01. Directorio administrativo de torneos.
-- Modelo: Torneo::obtenerDirectorioAdmin()
SELECT t.id,
       t.nombre,
       t.estado,
       t.formato,
       t.fecha_inicio,
       j.nombre AS nombre_juego,
       u.nombre_completo AS organizador_nombre
FROM torneos t
INNER JOIN juegos j ON t.juego_id = j.id
INNER JOIN usuarios u ON t.organizador_id = u.id
ORDER BY t.id DESC;

-- E02. Directorio administrativo filtrado por juego.
-- Modelo: Torneo::obtenerDirectorioAdmin($juegoId, null, 'nuevos')
SELECT t.id,
       t.nombre,
       t.estado,
       t.formato,
       t.fecha_inicio,
       j.nombre AS nombre_juego,
       u.nombre_completo AS organizador_nombre
FROM torneos t
INNER JOIN juegos j ON t.juego_id = j.id
INNER JOIN usuarios u ON t.organizador_id = u.id
WHERE t.juego_id = 1
ORDER BY t.id DESC;

-- E03. Directorio administrativo filtrado por estado.
-- Modelo: Torneo::obtenerDirectorioAdmin(null, $estado, 'nuevos')
SELECT t.id,
       t.nombre,
       t.estado,
       t.formato,
       t.fecha_inicio,
       j.nombre AS nombre_juego,
```

```sql
       u.nombre_completo AS organizador_nombre
FROM torneos t
INNER JOIN juegos j ON t.juego_id = j.id
INNER JOIN usuarios u ON t.organizador_id = u.id
WHERE t.estado = 'borrador'
ORDER BY t.id DESC;

-- E04. Ficha administrativa completa de un torneo.
-- Modelo: Torneo::obtenerDetalleAdmin()
SELECT t.*,
       j.nombre AS nombre_juego,
       j.categoria AS categoria_juego,
       m.nombre AS nombre_modalidad,
       m.descripcion AS descripcion_modalidad,
       sp.nombre AS nombre_sistema_puntuacion,
       u.nombre_completo AS organizador_nombre,
       u.email AS organizador_email,
       u.foto_perfil_url AS organizador_foto
FROM torneos t
INNER JOIN juegos j ON j.id = t.juego_id
LEFT JOIN modalidades m ON m.id = t.modalidad_id
LEFT JOIN sistemas_puntuacion sp ON sp.id = t.sistema_puntuacion_id
INNER JOIN usuarios u ON u.id = t.organizador_id
WHERE t.id = 1200
LIMIT 1;

-- E05. Participantes de un torneo en todos los estados.
-- Modelo: Torneo::obtenerParticipantesAdmin()
SELECT p.id,
       p.tipo,
       p.referencia_id,
       p.nombre,
       p.estado,
       p.fecha_inscripcion,
       p.fecha_confirmacion,
       CASE WHEN p.tipo = 'equipo' THEN e.escudo_url ELSE u.foto_perfil_url END AS
imagen,
       e.nombre_equipo AS equipo_nombre,
       CASE WHEN p.tipo = 'equipo'
            THEN (SELECT COUNT(*)
                  FROM equipo_miembros m
                  WHERE m.equipo_id = p.referencia_id AND m.es_activo = 1)
            ELSE NULL END AS total_miembros,
       COALESCE(pos.partidos_jugados, 0) AS partidos_jugados,
       COALESCE(pos.partidos_ganados, 0) AS partidos_ganados,
       COALESCE(pos.partidos_empatados, 0) AS partidos_empatados,
       COALESCE(pos.partidos_perdidos, 0) AS partidos_perdidos,
       COALESCE(pos.puntos, 0) AS puntos
FROM participantes_torneo p
LEFT JOIN equipos e ON p.tipo = 'equipo' AND e.id = p.referencia_id
LEFT JOIN usuarios u ON p.tipo = 'usuario' AND u.id = p.referencia_id
LEFT JOIN torneo_posiciones pos
       ON pos.participante_id = p.id
      AND pos.torneo_id = p.torneo_id
```

```sql
WHERE p.torneo_id = 1202
ORDER BY CASE p.estado
             WHEN 'confirmado' THEN 0
             WHEN 'pendiente' THEN 1
             WHEN 'rechazado' THEN 2
             ELSE 3
         END,
         COALESCE(pos.puntos, 0) DESC,
         p.nombre ASC;

-- E06. Tabla de posiciones administrativa de un torneo.
-- Modelo: Torneo::obtenerPosicionesAdmin()
SELECT p.nombre,
       COALESCE(pos.partidos_jugados, 0) AS partidos_jugados,
       COALESCE(pos.partidos_ganados, 0) AS partidos_ganados,
       COALESCE(pos.partidos_empatados, 0) AS partidos_empatados,
       COALESCE(pos.partidos_perdidos, 0) AS partidos_perdidos,
       COALESCE(pos.puntos_favor, 0) AS puntos_favor,
       COALESCE(pos.puntos_contra, 0) AS puntos_contra,
       COALESCE(pos.diferencia_goles, 0) AS diferencia_goles,
       COALESCE(pos.puntos, 0) AS puntos
FROM participantes_torneo p
LEFT JOIN torneo_posiciones pos
       ON pos.participante_id = p.id
      AND pos.torneo_id = p.torneo_id
WHERE p.torneo_id = 1202
  AND p.estado = 'confirmado'
ORDER BY COALESCE(pos.puntos, 0) DESC,
         COALESCE(pos.diferencia_goles, 0) DESC,
         COALESCE(pos.puntos_favor, 0) DESC,
         p.nombre ASC;

-- E07. Actividades de un torneo.
-- Modelo: Torneo::obtenerActividadesAdmin()
SELECT id, titulo, descripcion, tipo, fecha, hora
FROM torneo_actividades
WHERE torneo_id = 1202
ORDER BY fecha ASC, hora IS NULL, hora ASC;

-- E08. Historial de estados de un torneo.
-- Extra
SELECT c.id,
       c.fecha_cambio,
       c.estado_anterior,
       c.estado_nuevo,
       u.nombre_completo AS usuario,
       c.motivo
FROM torneo_cambio_estado c
INNER JOIN usuarios u ON u.id = c.usuario_id
WHERE c.torneo_id = 1200
ORDER BY c.fecha_cambio, c.id;

-- E09. Ocupacion de todos los torneos.
-- Extra
```

```sql
SELECT t.id,
       t.nombre,
       t.estado,
       t.cupo_max_equipos,
       COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END) AS confirmados,
       COUNT(CASE WHEN p.estado = 'pendiente' THEN 1 END) AS pendientes,
       ROUND(
           COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END)
           / t.cupo_max_equipos * 100, 1
       ) AS ocupacion_pct
FROM torneos t
LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
GROUP BY t.id, t.nombre, t.estado, t.cupo_max_equipos
ORDER BY ocupacion_pct DESC, t.id;

-- E10. Inscripciones pendientes por torneo.
-- Extra
SELECT t.id,
       t.nombre,
       t.estado,
       COUNT(p.id) AS inscripciones_pendientes
FROM torneos t
LEFT JOIN participantes_torneo p
       ON p.torneo_id = t.id
      AND p.estado = 'pendiente'
GROUP BY t.id, t.nombre, t.estado
HAVING COUNT(p.id) > 0
ORDER BY inscripciones_pendientes DESC;

-- E11. Torneos sin participantes.
-- Extra
SELECT t.id, t.nombre, t.estado
FROM torneos t
LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
WHERE p.id IS NULL
ORDER BY t.id;

-- E12. Torneos en curso o finalizados sin fixture.
-- Extra
SELECT t.id, t.nombre, t.estado
FROM torneos t
LEFT JOIN torneo_encuentros e ON e.torneo_id = t.id
WHERE t.estado IN ('en_curso', 'finalizado')
  AND e.id IS NULL;

-- E13. Torneos finalizados con encuentros sin finalizar.
-- Extra
SELECT t.id,
       t.nombre,
       COUNT(*) AS encuentros_sin_finalizar
FROM torneos t
INNER JOIN torneo_encuentros e ON e.torneo_id = t.id
WHERE t.estado = 'finalizado'
  AND e.estado <> 'finalizado'
```

```sql
GROUP BY t.id, t.nombre;

-- E14. Torneos finalizados sin podio guardado.
-- Extra
SELECT t.id, t.nombre, t.fecha_fin
FROM torneos t
LEFT JOIN torneo_config c
       ON c.torneo_id = t.id
      AND c.clave = 'podio'
WHERE t.estado = 'finalizado'
  AND c.torneo_id IS NULL;

-- E15. Impacto de eliminar un torneo (conteo de registros dependientes).
-- Extra (los borrados reales se hacen con las FK ON DELETE CASCADE)
SELECT t.id,
       t.nombre,
       (SELECT COUNT(*) FROM participantes_torneo WHERE torneo_id = t.id) AS
participantes,
       (SELECT COUNT(*) FROM torneo_encuentros WHERE torneo_id = t.id) AS
encuentros,
       (SELECT COUNT(*) FROM torneo_posiciones WHERE torneo_id = t.id) AS
posiciones,
       (SELECT COUNT(*) FROM torneo_suizo_parejas WHERE torneo_id = t.id) AS
parejas_suizo,
       (SELECT COUNT(*) FROM torneo_actividades WHERE torneo_id = t.id) AS
actividades,
       (SELECT COUNT(*) FROM torneo_comentarios WHERE torneo_id = t.id) AS
comentarios,
       (SELECT COUNT(*) FROM torneo_config WHERE torneo_id = t.id) AS
configuraciones,
       (SELECT COUNT(*) FROM torneo_cambio_estado WHERE torneo_id = t.id) AS
cambios_estado,
       (SELECT COUNT(*) FROM guardados
        WHERE tipo = 'torneo' AND referencia_id = t.id) AS guardados
FROM torneos t
WHERE t.id = 1200;

-- E16. Buscar torneos por nombre.
-- Extra
SELECT t.id,
       t.nombre,
       t.estado,
       j.nombre AS juego,
       u.nombre_completo AS organizador
FROM torneos t
INNER JOIN juegos j ON j.id = t.juego_id
INNER JOIN usuarios u ON u.id = t.organizador_id
WHERE t.nombre LIKE '%copa%'
ORDER BY t.id DESC;

-- E17. Torneos agrupados por organizador, incluyendo usuarios que ya no son rol
2.
-- Extra
SELECT t.organizador_id,
```

```sql
       u.nombre_completo AS organizador,
       u.rol_id,
       r.nombre_rol,
       COUNT(t.id) AS torneos
FROM torneos t
INNER JOIN usuarios u ON u.id = t.organizador_id
INNER JOIN roles r ON r.id = u.rol_id
GROUP BY t.organizador_id, u.nombre_completo, u.rol_id, r.nombre_rol
ORDER BY torneos DESC;

-- E18. Pivote de torneos por juego y estado.
-- Extra
SELECT j.id,
       j.nombre AS juego,
       COUNT(t.id) AS total,
       SUM(t.estado = 'borrador') AS borradores,
       SUM(t.estado = 'inscripciones_abiertas') AS abiertos,
       SUM(t.estado = 'en_curso') AS en_curso,
       SUM(t.estado = 'finalizado') AS finalizados,
       SUM(t.estado = 'cancelado') AS cancelados
FROM juegos j
LEFT JOIN torneos t ON t.juego_id = j.id
GROUP BY j.id, j.nombre
ORDER BY total DESC, j.nombre;

-- E19. Torneos con mas participantes confirmados.
-- Extra
SELECT t.id,
       t.nombre,
       t.estado,
       t.cupo_max_equipos,
       COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END) AS confirmados
FROM torneos t
LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
GROUP BY t.id, t.nombre, t.estado, t.cupo_max_equipos
ORDER BY confirmados DESC, t.id;

-- E20. Historial administrativo de torneos finalizados o cancelados.
-- Modelo: Torneo::obtenerHistorialTorneos()
SELECT t.id,
       t.nombre AS torneo_nombre,
       t.estado,
       t.formato,
       t.fecha_inicio,
       t.fecha_fin,
       j.nombre AS juego_nombre,
       u.nombre_completo AS organizador_nombre,
       (SELECT COUNT(*)
        FROM participantes_torneo
        WHERE torneo_id = t.id AND estado = 'confirmado') AS total_participantes
FROM torneos t
INNER JOIN juegos j ON j.id = t.juego_id
INNER JOIN usuarios u ON t.organizador_id = u.id
WHERE t.estado IN ('finalizado', 'cancelado')
```

```sql
ORDER BY t.fecha_fin DESC, t.id DESC;

-- E21. Catalogo completo de juegos (activos e inactivos).
-- Modelo: Juego::obtenerTodosLosJuegos()
SELECT *
FROM juegos
ORDER BY nombre ASC;

-- E22. Modalidades permitidas por un juego.
-- Modelo: Juego::obtenerModalidadesPermitidas()
SELECT m.id, m.nombre
FROM modalidades m
INNER JOIN juego_modalidades_permitidas jmp ON m.id = jmp.modalidad_id
WHERE jmp.juego_id = 1;

-- E23. Formatos permitidos por un juego.
-- Modelo: Juego::obtenerFormatosPermitidos()
SELECT f.codigo, f.nombre, f.activo
FROM formatos_torneo f
INNER JOIN juego_formatos_permitidos jfp ON f.codigo = jfp.formato_codigo
WHERE jfp.juego_id = 1
  AND f.activo = 1;

-- E24. Todos los formatos con su estado.
-- Modelo: Formato::obtenerTodas()
SELECT codigo, nombre, descripcion, activo
FROM formatos_torneo
ORDER BY codigo;

-- E25. Torneos con fechas incoherentes (la tabla tiene un CHECK, pero se audita).
-- Extra
SELECT id, nombre, fecha_inicio, fecha_fin
FROM torneos
WHERE fecha_inicio IS NOT NULL
  AND fecha_fin IS NOT NULL
  AND fecha_inicio > fecha_fin;

-- E26. Torneos abiertos con la fecha limite de inscripcion vencida.
-- Extra
SELECT id, nombre, fecha_limite_inscripcion, fecha_inicio
FROM torneos
WHERE estado = 'inscripciones_abiertas'
  AND fecha_limite_inscripcion IS NOT NULL
  AND fecha_limite_inscripcion < CURDATE();

-- E27. Comentarios pendientes de moderacion.
-- Extra
SELECT c.id,
       c.torneo_id,
       t.nombre AS torneo,
       c.nombre_autor,
       c.email_autor,
       c.comentario,
       c.fecha_hora
```

```sql
FROM torneo_comentarios c
INNER JOIN torneos t ON t.id = c.torneo_id
WHERE c.aprobado = 0
ORDER BY c.fecha_hora DESC;

-- E28. Todas las configuraciones guardadas de un torneo.
-- Extra
SELECT clave, valor
FROM torneo_config
WHERE torneo_id = 1200
ORDER BY clave;

-- E29. Torneos con el cupo maximo excedido.
-- Extra
SELECT t.id,
       t.nombre,
       t.cupo_max_equipos,
       COUNT(CASE WHEN p.estado = 'confirmado' THEN 1 END) AS confirmados
FROM torneos t
INNER JOIN participantes_torneo p ON p.torneo_id = t.id
GROUP BY t.id, t.nombre, t.cupo_max_equipos
HAVING confirmados > t.cupo_max_equipos;

-- ============================================================================
-- BLOQUE F - AUDITORIA, LOGS, SESIONES Y ERRORES
-- ============================================================================

-- F01. Ultimos movimientos de la auditoria de cambios.
-- Extra (no se seleccionan datos_viejos/datos_nuevos para no exponer datos
-- sensibles; se pueden consultar puntualmente en F03)
SELECT a.id,
       a.tabla_afectada,
       a.registro_id,
       a.accion,
       a.usuario_id,
       u.email AS usuario_email,
       a.ip_origen,
       a.fecha_hora
FROM auditoria_cambios a
LEFT JOIN usuarios u ON u.id = a.usuario_id
ORDER BY a.fecha_hora DESC, a.id DESC
LIMIT 100;

-- F02. Movimientos de auditoria por tabla.
-- Extra
SELECT tabla_afectada,
       accion,
       COUNT(*) AS total,
       MIN(fecha_hora) AS primer_movimiento,
       MAX(fecha_hora) AS ultimo_movimiento
FROM auditoria_cambios
GROUP BY tabla_afectada, accion
ORDER BY total DESC;
```

```sql
-- F03. Auditoria de un registro concreto.
-- Extra
SELECT a.id,
       a.tabla_afectada,
       a.registro_id,
       a.accion,
       a.usuario_id,
       u.email AS usuario_email,
       a.fecha_hora
FROM auditoria_cambios a
LEFT JOIN usuarios u ON u.id = a.usuario_id
WHERE a.tabla_afectada = 'torneos'
  AND a.registro_id = 1200
ORDER BY a.fecha_hora DESC, a.id DESC;

-- F04. Movimientos de auditoria realizados por un usuario.
-- Extra
SELECT a.id,
       a.tabla_afectada,
       a.registro_id,
       a.accion,
       a.fecha_hora
FROM auditoria_cambios a
WHERE a.usuario_id = 1
ORDER BY a.fecha_hora DESC, a.id DESC;

-- F05. Actividad reciente con correo del usuario.
-- Modelo: Auditoria::obtenerTodos()
SELECT l.*,
       u.email AS usuario_email
FROM logs_actividad l
LEFT JOIN usuarios u ON l.usuario_id = u.id
ORDER BY l.fecha_hora DESC
LIMIT 100;

-- F06. Actividad agrupada por accion.
-- Extra
SELECT accion,
       COUNT(*) AS total,
       COUNT(DISTINCT usuario_id) AS usuarios_distintos,
       MIN(fecha_hora) AS primera_vez,
       MAX(fecha_hora) AS ultima_vez
FROM logs_actividad
GROUP BY accion
ORDER BY total DESC;

-- F07. Ultimos accesos al sistema.
-- Extra
SELECT l.id,
       l.email_intentado,
       l.usuario_id,
       u.nombre_completo AS usuario,
       l.exito,
       l.ip_origen,
```

```
       l.mensaje_error,
       l.fecha_hora
FROM logs_acceso l
LEFT JOIN usuarios u ON u.id = l.usuario_id
ORDER BY l.fecha_hora DESC, l.id DESC
LIMIT 100;

-- F08. Fallos de acceso en las ultimas 24 horas.
-- Extra
SELECT id,
       email_intentado,
       usuario_id,
       ip_origen,
       user_agent,
       mensaje_error,
       fecha_hora
FROM logs_acceso
WHERE exito = 0
  AND fecha_hora >= NOW() - INTERVAL 24 HOUR
ORDER BY fecha_hora DESC;

-- F09. Correos con mas fallos de acceso.
-- Extra
SELECT email_intentado,
       COUNT(*) AS fallos,
       COUNT(DISTINCT ip_origen) AS ips_distintas,
       MAX(fecha_hora) AS ultimo_fallo
FROM logs_acceso
WHERE exito = 0
  AND email_intentado IS NOT NULL
GROUP BY email_intentado
ORDER BY fallos DESC
LIMIT 20;

-- F10. IPs con mas fallos de acceso.
-- Extra
SELECT ip_origen,
       COUNT(*) AS fallos,
       COUNT(DISTINCT email_intentado) AS correos_distintos,
       MAX(fecha_hora) AS ultimo_fallo
FROM logs_acceso
WHERE exito = 0
GROUP BY ip_origen
ORDER BY fallos DESC
LIMIT 20;

-- F11. IPs actualmente bloqueadas por rate limiting.
-- Modelo: SeguridadAuth::estaIpBloqueada()
SELECT ip_origen,
       intentos,
       ultimo_intento,
       bloqueado_hasta
FROM intentos_login
WHERE bloqueado_hasta IS NOT NULL
```

```sql
  AND bloqueado_hasta > NOW()
ORDER BY bloqueado_hasta DESC;

-- F12. Sesiones activas de todos los usuarios.
-- Extra
SELECT s.id,
       s.usuario_id,
       u.nombre_completo AS usuario,
       u.email,
       s.ip_origen,
       s.ultima_actividad,
       s.creado_en,
       s.expira_en
FROM sesiones_activas s
INNER JOIN usuarios u ON u.id = s.usuario_id
WHERE s.activa = 1
ORDER BY s.ultima_actividad DESC;

-- F13. Sesiones marcadas activas pero vencidas.
-- Extra
SELECT s.id,
       s.usuario_id,
       u.nombre_completo AS usuario,
       s.expira_en,
       s.ultima_actividad
FROM sesiones_activas s
INNER JOIN usuarios u ON u.id = s.usuario_id
WHERE s.activa = 1
  AND s.expira_en < NOW()
ORDER BY s.expira_en;

-- F14. Cantidad de sesiones por usuario.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       COUNT(s.id) AS sesiones_totales,
       SUM(s.activa = 1) AS sesiones_activas
FROM usuarios u
LEFT JOIN sesiones_activas s ON s.usuario_id = u.id
GROUP BY u.id, u.nombre_completo, u.email
HAVING sesiones_totales > 0
ORDER BY sesiones_activas DESC;

-- F15. Errores agrupados con filtros del panel.
-- Modelo: ErrorSistema::obtenerErroresAgrupados()
SELECT e.id,
       e.hash_error,
       e.tipo_error,
       e.mensaje,
       e.archivo,
       e.linea,
       e.cantidad_ocurrencias,
       e.primera_vez,
```

```sql
       e.ultima_vez,
       e.estado,
       u.email AS usuario_email,
       e.rol_usuario
FROM errores_sistema e
LEFT JOIN usuarios u ON e.usuario_id = u.id
ORDER BY FIELD(e.estado, 'nuevo', 'revision', 'resuelto'), e.ultima_vez DESC;

-- F16. Errores filtrados por estado, rol y tipo.
-- Modelo: ErrorSistema::obtenerErroresAgrupados('nuevo', '3', 'codigo')
SELECT e.id,
       e.hash_error,
       e.tipo_error,
       e.mensaje,
       e.archivo,
       e.linea,
       e.cantidad_ocurrencias,
       e.primera_vez,
       e.ultima_vez,
       e.estado,
       u.email AS usuario_email,
       e.rol_usuario
FROM errores_sistema e
LEFT JOIN usuarios u ON e.usuario_id = u.id
WHERE e.estado = 'nuevo'
  AND e.rol_usuario = '3'
  AND e.tipo_error = 'codigo'
ORDER BY e.ultima_vez DESC;

-- F17. Errores con mas ocurrencias.
-- Extra
SELECT id,
       hash_error,
       tipo_error,
       mensaje,
       archivo,
       linea,
       cantidad_ocurrencias,
       estado,
       primera_vez,
       ultima_vez
FROM errores_sistema
ORDER BY cantidad_ocurrencias DESC
LIMIT 20;

-- F18. Detalle de un error.
-- Modelo: ErrorSistema::obtenerDetalleError()
SELECT e.*,
       u.email AS usuario_email
FROM errores_sistema e
LEFT JOIN usuarios u ON e.usuario_id = u.id
WHERE e.id = 1;

-- F19. Resumen de salud del sistema.
```

```sql
-- Extra
SELECT (SELECT COUNT(*) FROM errores_sistema WHERE estado = 'nuevo') AS
errores_nuevos,
       (SELECT COUNT(*) FROM errores_sistema WHERE estado = 'revision') AS
errores_en_revision,
       (SELECT COUNT(*) FROM logs_acceso
        WHERE exito = 0 AND fecha_hora >= NOW() - INTERVAL 24 HOUR) AS
fallos_login_24h,
       (SELECT COUNT(*) FROM intentos_login
        WHERE bloqueado_hasta IS NOT NULL AND bloqueado_hasta > NOW()) AS
ips_bloqueadas,
       (SELECT COUNT(*) FROM sesiones_activas WHERE activa = 1) AS
sesiones_activas,
       (SELECT COUNT(*) FROM sesiones_activas
        WHERE activa = 1 AND expira_en < NOW()) AS sesiones_vencidas_activas;

-- F20. Intentos fallidos de ingreso por codigo de equipo en la ultima hora.
-- Modelo: Equipo::estaBloqueadoPorIntentosDeCodigo()
SELECT usuario_id,
       COUNT(*) AS intentos_fallidos,
       MIN(fecha_hora) AS primero,
       MAX(fecha_hora) AS ultimo
FROM logs_actividad
WHERE accion = 'UNION_CODIGO_FALLIDA'
  AND fecha_hora >= NOW() - INTERVAL 1 HOUR
GROUP BY usuario_id
ORDER BY intentos_fallidos DESC;

-- F21. Diccionario de datos de la base ascend.
-- Extra
SELECT table_name AS tabla,
       column_name AS columna,
       column_type AS tipo,
       is_nullable AS permite_nulo,
       column_key AS clave,
       column_default AS valor_defecto
FROM information_schema.columns
WHERE table_schema = 'ascend'
ORDER BY table_name, ordinal_position;

-- F22. Claves foraneas del esquema.
-- Extra
SELECT table_name AS tabla_origen,
       column_name AS columna_origen,
       referenced_table_name AS tabla_destino,
       referenced_column_name AS columna_destino
FROM information_schema.key_column_usage
WHERE table_schema = 'ascend'
  AND referenced_table_name IS NOT NULL
ORDER BY table_name, column_name;

-- F23. Tamano aproximado de las tablas.
-- Extra
SELECT table_name AS tabla,
```

```sql
       table_rows AS filas_aproximadas,
       ROUND((data_length + index_length) / 1024 / 1024, 2) AS mb
FROM information_schema.tables
WHERE table_schema = 'ascend'
ORDER BY (data_length + index_length) DESC;

-- F24. Indices definidos en la base.
-- Extra
SELECT table_name AS tabla,
       index_name AS indice,
       GROUP_CONCAT(column_name ORDER BY seq_in_index SEPARATOR ', ') AS columnas,
       non_unique AS no_unico
FROM information_schema.statistics
WHERE table_schema = 'ascend'
GROUP BY table_name, index_name, non_unique
ORDER BY table_name, index_name;

-- ============================================================================
-- BLOQUE G - COMUNICACIONES: NOTIFICACIONES, NEWSLETTER Y COMENTARIOS
-- ============================================================================

-- G01. Ultimas notificaciones del sistema con destinatario.
-- Extra
SELECT n.id,
       n.usuario_id,
       u.nombre_completo AS destinatario,
       u.email,
       n.tipo,
       n.titulo,
       n.mensaje,
       n.leido,
       n.creado_en
FROM notificaciones n
INNER JOIN usuarios u ON u.id = n.usuario_id
ORDER BY n.creado_en DESC, n.id DESC
LIMIT 100;

-- G02. Notificaciones no leidas por usuario.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       COUNT(n.id) AS no_leidas
FROM usuarios u
INNER JOIN notificaciones n ON n.usuario_id = u.id AND n.leido = 0
GROUP BY u.id, u.nombre_completo, u.email
ORDER BY no_leidas DESC;

-- G03. Notificaciones por tipo y estado de lectura.
-- Extra
SELECT tipo,
       COUNT(*) AS total,
       SUM(leido = 1) AS leidas,
       SUM(leido = 0) AS no_leidas
```

```sql
FROM notificaciones
GROUP BY tipo
ORDER BY total DESC;

-- G04. Audiencia de una alerta global: jugadores y organizadores activos.
-- Modelo: Alerta::enviarAlertaGlobal() con audiencia 'todos'
SELECT id, nombre_completo, email, rol_id
FROM usuarios
WHERE esta_activo = 1
  AND rol_id IN (2, 3)
ORDER BY rol_id, nombre_completo;

-- G05. Audiencia de una alerta para jugadores.
-- Modelo: Alerta::enviarAlertaGlobal() con audiencia 'jugador'
SELECT id, nombre_completo, email
FROM usuarios
WHERE esta_activo = 1
  AND rol_id = 3
ORDER BY nombre_completo;

-- G06. Audiencia de una alerta para organizadores.
-- Modelo: Alerta::enviarAlertaGlobal() con audiencia 'organizador'
SELECT id, nombre_completo, email
FROM usuarios
WHERE esta_activo = 1
  AND rol_id = 2
ORDER BY nombre_completo;

-- G07. Administradores activos que reciben los mensajes de contacto.
-- Modelo: MensajeContacto::enviarAAdministradores()
SELECT id, nombre_completo, email
FROM usuarios
WHERE rol_id = 1
  AND esta_activo = 1
ORDER BY id;

-- G08. Suscriptores activos del newsletter.
-- Modelo: Newsletter::listarActivos()
SELECT id, email, fecha_suscripcion
FROM newsletter_subscriptores
WHERE activo = 1
ORDER BY fecha_suscripcion DESC, id DESC
LIMIT 100;

-- G09. Suscriptores inactivos y total historico.
-- Extra
SELECT activo,
       COUNT(*) AS total,
       MIN(fecha_suscripcion) AS primer_suscriptor,
       MAX(fecha_suscripcion) AS ultimo_suscriptor
FROM newsletter_subscriptores
GROUP BY activo
ORDER BY activo DESC;
```

```sql
-- G10. Comentarios pendientes de aprobacion.
-- Extra
SELECT c.id,
       c.torneo_id,
       t.nombre AS torneo,
       c.nombre_autor,
       c.email_autor,
       c.comentario,
       c.fecha_hora
FROM torneo_comentarios c
INNER JOIN torneos t ON t.id = c.torneo_id
WHERE c.aprobado = 0
ORDER BY c.fecha_hora DESC;

-- G11. Cantidad de comentarios por torneo.
-- Extra
SELECT t.id,
       t.nombre AS torneo,
       COUNT(c.id) AS total_comentarios,
       SUM(c.aprobado = 1) AS aprobados,
       SUM(c.aprobado = 0) AS pendientes
FROM torneos t
LEFT JOIN torneo_comentarios c ON c.torneo_id = t.id
GROUP BY t.id, t.nombre
HAVING COUNT(c.id) > 0
ORDER BY total_comentarios DESC;

-- G12. Comentarios mas recientes con autor registrado.
-- Extra
SELECT c.id,
       c.torneo_id,
       t.nombre AS torneo,
       COALESCE(u.nombre_completo, c.nombre_autor) AS autor,
       c.comentario,
       c.aprobado,
       c.fecha_hora
FROM torneo_comentarios c
INNER JOIN torneos t ON t.id = c.torneo_id
LEFT JOIN usuarios u ON u.id = c.usuario_id
ORDER BY c.fecha_hora DESC, c.id DESC
LIMIT 50;

-- G13. Elementos guardados por tipo.
-- Extra
SELECT tipo,
       COUNT(*) AS total_guardados,
       COUNT(DISTINCT usuario_id) AS usuarios
FROM guardados
GROUP BY tipo
ORDER BY total_guardados DESC;

-- G14. Notificaciones generadas por tipo de alerta en los ultimos 30 dias.
-- Extra
SELECT DATE(creado_en) AS dia,
```

```
       tipo,
       COUNT(*) AS total
FROM notificaciones
WHERE creado_en >= CURDATE() - INTERVAL 30 DAY
GROUP BY DATE(creado_en), tipo
ORDER BY dia DESC, total DESC;

-- ============================================================================
-- BLOQUE H - INTEGRIDAD, CONTROL DE ROLES Y CIERRE DE DEFENSA
-- ============================================================================

-- H01. Cuentas de Administrador General y su estado.
-- Extra
SELECT id,
       nombre_completo,
       email,
       esta_activo,
       email_verificado,
       fecha_registro,
       ultimo_acceso
FROM usuarios
WHERE rol_id = 1
ORDER BY id;

-- H02. Cantidad de cuentas por rol (incluye roles no asignables).
-- Extra
SELECT r.id,
       r.nombre_rol,
       COUNT(u.id) AS total
FROM roles r
LEFT JOIN usuarios u ON u.rol_id = r.id
GROUP BY r.id, r.nombre_rol
ORDER BY r.id;

-- H03. Cuentas con rol 4 Usuario Publico (debe devolver cero filas).
-- Extra
SELECT id, nombre_completo, email, esta_activo, fecha_registro
FROM usuarios
WHERE rol_id = 4;

-- H04. Cuentas con rol 1 creadas o modificadas recientemente.
-- Extra (el alta de administradores se hace fuera de la aplicacion)
SELECT id, nombre_completo, email, fecha_registro, ultimo_acceso
FROM usuarios
WHERE rol_id = 1
  AND fecha_registro >= CURDATE() - INTERVAL 30 DAY
ORDER BY fecha_registro DESC;

-- H05. Organizadores que crearon equipos.
-- Extra (auditoria de datos; el flujo web solo permite crear equipos a jugadores)
SELECT u.id AS creador_id,
       u.nombre_completo,
       u.rol_id,
       COUNT(e.id) AS equipos_creados,
```

```sql
       GROUP_CONCAT(e.nombre_equipo ORDER BY e.id SEPARATOR ' | ') AS equipos
FROM equipos e
INNER JOIN usuarios u ON u.id = e.creado_por
WHERE u.rol_id IN (1, 2)
GROUP BY u.id, u.nombre_completo, u.rol_id
ORDER BY equipos_creados DESC;

-- H06. Detalle de equipos creados por usuarios que no son jugadores.
-- Extra
SELECT e.id,
       e.nombre_equipo,
       e.fecha_creacion,
       e.activo,
       u.id AS creador_id,
       u.nombre_completo AS creador,
       r.nombre_rol
FROM equipos e
INNER JOIN usuarios u ON u.id = e.creado_por
INNER JOIN roles r ON r.id = u.rol_id
WHERE u.rol_id <> 3
ORDER BY e.fecha_creacion DESC;

-- H07. Usuarios cuyo rol no coincide con sus perfiles.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.rol_id,
       r.nombre_rol,
       pj.usuario_id AS tiene_perfil_jugador,
       po.usuario_id AS tiene_perfil_organizador
FROM usuarios u
INNER JOIN roles r ON r.id = u.rol_id
LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
WHERE (u.rol_id = 3 AND pj.usuario_id IS NULL)
   OR (u.rol_id = 2 AND po.usuario_id IS NULL)
   OR (u.rol_id NOT IN (2, 3)
       AND (pj.usuario_id IS NOT NULL OR po.usuario_id IS NOT NULL))
ORDER BY u.id;

-- H08. Capitanes que no son miembros activos.
-- Extra
SELECT ec.equipo_id,
       e.nombre_equipo,
       u.id AS capitan_id,
       u.nombre_completo AS capitan,
       ec.es_capitan_principal
FROM equipo_capitanes ec
INNER JOIN equipos e ON e.id = ec.equipo_id
INNER JOIN usuarios u ON u.id = ec.usuario_id
LEFT JOIN equipo_miembros em
       ON em.equipo_id = ec.equipo_id
      AND em.usuario_id = ec.usuario_id
WHERE em.usuario_id IS NULL
```

```sql
   OR em.es_activo = 0
ORDER BY ec.equipo_id;

-- H09. Equipos con mas de un capitan principal.
-- Extra
SELECT equipo_id,
       COUNT(*) AS capitanes_principales
FROM equipo_capitanes
WHERE es_capitan_principal = 1
GROUP BY equipo_id
HAVING COUNT(*) > 1;

-- H10. Participantes que apuntan a equipos o usuarios inactivos.
-- Extra
SELECT p.id,
       p.torneo_id,
       p.tipo,
       p.referencia_id,
       p.nombre,
       p.estado,
       CASE WHEN p.tipo = 'equipo' THEN e.activo ELSE u.esta_activo END AS
referencia_activa
FROM participantes_torneo p
LEFT JOIN equipos e ON p.tipo = 'equipo' AND e.id = p.referencia_id
LEFT JOIN usuarios u ON p.tipo = 'usuario' AND u.id = p.referencia_id
WHERE (p.tipo = 'equipo' AND (e.id IS NULL OR e.activo = 0))
   OR (p.tipo = 'usuario' AND (u.id IS NULL OR u.esta_activo = 0));

-- H11. Conflicto: jugador en dos equipos del mismo torneo.
-- Extra
SELECT t.id AS torneo_id,
       t.nombre AS torneo,
       u.id AS usuario_id,
       u.nombre_completo,
       COUNT(DISTINCT e.id) AS equipos
FROM usuarios u
INNER JOIN equipo_miembros em ON em.usuario_id = u.id AND em.es_activo = 1
INNER JOIN equipos e ON e.id = em.equipo_id AND e.activo = 1
INNER JOIN participantes_torneo p
        ON p.tipo = 'equipo'
       AND p.referencia_id = e.id
       AND p.estado IN ('confirmado', 'pendiente')
INNER JOIN torneos t
        ON t.id = p.torneo_id
       AND t.estado IN ('inscripciones_abiertas', 'en_curso')
GROUP BY t.id, t.nombre, u.id, u.nombre_completo
HAVING COUNT(DISTINCT e.id) > 1
ORDER BY t.id, u.nombre_completo;

-- H12. Jugadores activos sin equipo.
-- Extra
SELECT u.id, u.nombre_completo, u.email
FROM usuarios u
LEFT JOIN equipo_miembros em ON em.usuario_id = u.id AND em.es_activo = 1
```

```sql
WHERE u.rol_id = 3
  AND u.esta_activo = 1
  AND em.usuario_id IS NULL
ORDER BY u.nombre_completo;

-- H13. Equipos activos sin participacion en torneos.
-- Extra
SELECT e.id, e.nombre_equipo
FROM equipos e
LEFT JOIN participantes_torneo p ON p.tipo = 'equipo' AND p.referencia_id = e.id
WHERE e.activo = 1
  AND p.id IS NULL
ORDER BY e.nombre_equipo;

-- H14. Torneos sin participantes.
-- Extra
SELECT t.id, t.nombre, t.estado
FROM torneos t
LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
WHERE p.id IS NULL
ORDER BY t.id;

-- H15. Torneos abiertos con inscripcion vencida.
-- Extra
SELECT id, nombre, fecha_limite_inscripcion
FROM torneos
WHERE estado = 'inscripciones_abiertas'
  AND fecha_limite_inscripcion IS NOT NULL
  AND fecha_limite_inscripcion < CURDATE();

-- H16. Torneos finalizados sin podio.
-- Extra
SELECT t.id, t.nombre
FROM torneos t
LEFT JOIN torneo_config c
       ON c.torneo_id = t.id
      AND c.clave = 'podio'
WHERE t.estado = 'finalizado'
  AND c.torneo_id IS NULL;

-- H17. Errores sin resolver.
-- Extra
SELECT id, tipo_error, mensaje, archivo, linea, cantidad_ocurrencias, estado,
ultima_vez
FROM errores_sistema
WHERE estado IN ('nuevo', 'revision')
ORDER BY ultima_vez DESC;

-- H18. Resumen de integridad en una sola fila.
-- Extra
SELECT (SELECT COUNT(*) FROM usuarios WHERE rol_id = 4) AS usuarios_rol_4,
       (SELECT COUNT(*) FROM usuarios WHERE rol_id = 1) AS administradores,
       (SELECT COUNT(*) FROM equipos e
        LEFT JOIN equipo_miembros em
```

```
                ON em.equipo_id = e.id AND em.es_activo = 1
         WHERE em.usuario_id IS NULL) AS equipos_sin_miembros,
        (SELECT COUNT(*) FROM equipos e
         LEFT JOIN equipo_capitanes ec ON ec.equipo_id = e.id
         WHERE ec.equipo_id IS NULL) AS equipos_sin_capitan,
        (SELECT COUNT(*) FROM usuarios u
         LEFT JOIN perfiles_jugadores pj ON pj.usuario_id = u.id
         WHERE u.rol_id = 3 AND pj.usuario_id IS NULL) AS jugadores_sin_perfil,
        (SELECT COUNT(*) FROM usuarios u
         LEFT JOIN perfiles_organizadores po ON po.usuario_id = u.id
         WHERE u.rol_id = 2 AND po.usuario_id IS NULL) AS organizadores_sin_perfil,
        (SELECT COUNT(*) FROM torneos t
         LEFT JOIN participantes_torneo p ON p.torneo_id = t.id
         WHERE t.estado IN ('en_curso', 'finalizado') AND p.id IS NULL) AS
torneos_sin_participantes,
        (SELECT COUNT(*) FROM torneos t
         LEFT JOIN torneo_encuentros e ON e.torneo_id = t.id
         WHERE t.estado IN ('en_curso', 'finalizado') AND e.id IS NULL) AS
torneos_sin_fixture;

-- H19. Equipos creados por organizador (detalle para la defensa).
-- Extra (responde a la pregunta de por que la semilla muestra equipos con
-- creadores que hoy son organizadores)
SELECT e.id AS equipo_id,
       e.nombre_equipo,
       e.fecha_creacion,
       e.activo,
       u.id AS creador_id,
       u.nombre_completo AS creador,
       u.rol_id,
       r.nombre_rol
FROM equipos e
INNER JOIN usuarios u ON u.id = e.creado_por
INNER JOIN roles r ON r.id = u.rol_id
WHERE u.rol_id IN (1, 2)
ORDER BY u.rol_id, e.fecha_creacion;

-- H20. Equipos creados por organizador y cantidad de torneos en los que
participan.
-- Extra
SELECT u.id AS creador_id,
       u.nombre_completo AS creador,
       COUNT(DISTINCT e.id) AS equipos,
       COUNT(DISTINCT p.torneo_id) AS torneos_con_esos_equipos,
       COUNT(p.id) AS participaciones
FROM usuarios u
INNER JOIN equipos e ON e.creado_por = u.id
LEFT JOIN participantes_torneo p
       ON p.tipo = 'equipo'
      AND p.referencia_id = e.id
WHERE u.rol_id IN (1, 2)
GROUP BY u.id, u.nombre_completo
ORDER BY equipos DESC;
```

```sql
-- H21. Torneos organizados por usuarios que ya no tienen rol Organizador.
-- Extra (control de cambios de rol)
SELECT u.id,
       u.nombre_completo,
       u.rol_id,
       r.nombre_rol,
       COUNT(t.id) AS torneos
FROM usuarios u
INNER JOIN roles r ON r.id = u.rol_id
INNER JOIN torneos t ON t.organizador_id = u.id
WHERE u.rol_id <> 2
GROUP BY u.id, u.nombre_completo, u.rol_id, r.nombre_rol
ORDER BY torneos DESC;

-- H22. Cuentas activas sin verificar correo pero con sesion activa.
-- Extra
SELECT u.id,
       u.nombre_completo,
       u.email,
       u.fecha_registro,
       COUNT(s.id) AS sesiones_activas
FROM usuarios u
INNER JOIN sesiones_activas s ON s.usuario_id = u.id AND s.activa = 1
WHERE u.esta_activo = 1
  AND u.email_verificado = 0
GROUP BY u.id, u.nombre_completo, u.email, u.fecha_registro
ORDER BY sesiones_activas DESC;

-- H23. Nombres de equipo duplicados (la columna es UNIQUE; debe devolver cero).
-- Extra
SELECT nombre_equipo, COUNT(*) AS repeticiones
FROM equipos
GROUP BY nombre_equipo
HAVING COUNT(*) > 1;

-- H24. Correos duplicados (la columna es UNIQUE; debe devolver cero).
-- Extra
SELECT email, COUNT(*) AS repeticiones
FROM usuarios
GROUP BY email
HAVING COUNT(*) > 1;

-- H25. Perfiles huerfanos (las FK lo impiden; control inverso).
-- Extra
SELECT 'perfiles_jugadores' AS tabla, pj.usuario_id
FROM perfiles_jugadores pj
LEFT JOIN usuarios u ON u.id = pj.usuario_id
WHERE u.id IS NULL
UNION ALL
SELECT 'perfiles_organizadores' AS tabla, po.usuario_id
FROM perfiles_organizadores po
LEFT JOIN usuarios u ON u.id = po.usuario_id
WHERE u.id IS NULL;
```

```
-- ============================================================================
-- ANEXO A - SENTENCIAS DE ESCRITURA DEL PANEL (REFERENCIA, NO EJECUTAR)
-- ============================================================================
-- Estas sentencias documentan las operaciones reales de administracion.
-- No se ejecutan al correr el archivo. Los valores con :nombre son parametros
-- de las consultas preparadas del backend.
--
-- Usuario::responderSolicitudOrganizador() / cambiarVerificacionOrganizador()
-- INSERT INTO perfiles_organizadores
--     (usuario_id, nombre_organizacion, verificado_oficial)
-- VALUES (:usuario_id, 'Organizador Independiente', :estado)
-- ON DUPLICATE KEY UPDATE verificado_oficial = VALUES(verificado_oficial);
--
-- Usuario::cambiarEstadoUsuario() / eliminarUsuario()
-- UPDATE usuarios SET esta_activo = :estado WHERE id = :id;
--
-- Usuario::actualizarUsuario()
-- UPDATE usuarios
-- SET nombre_completo = :nombre_completo, rol_id = :rol
-- WHERE id = :id;
-- INSERT INTO perfiles_organizadores
--     (usuario_id, nombre_organizacion, verificado_oficial)
-- SELECT id, 'Organizador Independiente', 0 FROM usuarios WHERE id = :id;
-- INSERT INTO perfiles_jugadores (usuario_id, apodo_gamertag)
-- SELECT id, LEFT(SUBSTRING_INDEX(email, '@', 1), 50)
-- FROM usuarios WHERE id = :id;
--
-- Equipo::renombrarEquipoAdmin()
-- UPDATE equipos SET nombre_equipo = :nombre WHERE id = :id;
-- UPDATE participantes_torneo
-- SET nombre = :nombre
-- WHERE tipo = 'equipo' AND referencia_id = :id;
-- UPDATE torneo_config SET valor = :valor
-- WHERE torneo_id = :torneo_id AND clave = 'podio';
--
-- Torneo::actualizarPorAdmin()
-- UPDATE torneos SET nombre = :nombre, estado = :estado WHERE id = :id;
--
-- Torneo::eliminarPorAdmin()
-- DELETE FROM guardados WHERE tipo = 'torneo' AND referencia_id = :id;
-- DELETE FROM torneos WHERE id = :id;
--
-- ErrorSistema::cambiarEstado() / eliminarError()
-- UPDATE errores_sistema
-- SET estado = :estado, detalles_resolucion = :detalles
-- WHERE id = :id;
-- DELETE FROM errores_sistema WHERE id = :id;
--
-- Juego::crearJuego() / actualizarJuego() / cambiarEstado()
-- INSERT INTO juegos
--     (nombre, categoria, imagen_url, puntos_victoria, puntos_empate,
--      puntos_derrota, activo)
-- VALUES (:nombre, :categoria, :imagen, :p_vic, :p_emp, :p_der, 1);
-- INSERT INTO juego_modalidades_permitidas (juego_id, modalidad_id)
```

```
-- VALUES (:juego_id, :modalidad_id);
-- INSERT INTO juego_formatos_permitidos (juego_id, formato_codigo)
-- VALUES (:juego_id, :formato_codigo);
-- UPDATE juegos SET activo = :activo WHERE id = :id;
--
-- Formato::cambiarEstado()
-- UPDATE formatos_torneo SET activo = :activo WHERE codigo = :codigo;
--
-- Alerta::enviarAlertaGlobal()
-- INSERT INTO notificaciones (usuario_id, tipo, titulo, mensaje)
-- VALUES (:usuario_id, :tipo, :titulo, :mensaje);
--
-- MensajeContacto::enviarAAdministradores()
-- INSERT INTO notificaciones (usuario_id, tipo, titulo, mensaje)
-- VALUES (:usuario_id, 'info', :titulo, :mensaje);
--
-- Newsletter::suscribir()
-- UPDATE newsletter_subscriptores SET activo = 1 WHERE id = :id;
-- INSERT INTO newsletter_subscriptores (email, activo) VALUES (:email, 1);
--
-- PoliticaContrasena::guardarPolitica()
-- UPDATE politicas_contrasenas
-- SET longitud_minima = :longitud_minima,
--     requiere_mayuscula = :requiere_mayuscula,
--     requiere_minuscula = :requiere_minuscula,
--     requiere_numero = :requiere_numero,
--     requiere_caracter_especial = :requiere_caracter_especial,
--     expiracion_dias = :expiracion_dias,
--     historial_cantidad = :historial_cantidad,
--     actualizado_por = :actualizado_por
-- WHERE id = 1;
--
-- SeguridadAuth::registrarIntentoFallido() / limpiarIntentos()
-- UPDATE intentos_login
-- SET intentos = :intentos, ultimo_intento = CURRENT_TIMESTAMP,
--     bloqueado_hasta = :bloqueado_hasta
-- WHERE ip_origen = :ip;
-- INSERT INTO intentos_login (ip_origen, intentos, ultimo_intento)
-- VALUES (:ip, 1, CURRENT_TIMESTAMP);
-- DELETE FROM intentos_login WHERE ip_origen = :ip;
-- ============================================================================
```